

Projet Informatique de 2ème année

Groupe 32

EX-Libris

Étudiants :

Arnaud BICHON
Axel LECLERCQ
Cécile LOCATELLI
Moussa SOULAMA
Anne-Camille TAMPÉ
Paul FOURCADE

Encadrant :

Elwenn JOUBREL

Table des matières

1 Introduction

Depuis plusieurs années, le numérique a profondément transformé la manière dont nous consommons et partageons nos loisirs culturels. Le cinéma dispose de Letterboxd, la musique de Last.fm : des plateformes communautaires qui permettent à chacun de garder une trace de ses découvertes, de les noter et d'échanger avec d'autres passionnés. La lecture, pourtant, reste étonnamment mal servie sur ce plan. Le principal service existant, Goodreads, appartient à Amazon, ce qui pose des questions en matière d'indépendance éditoriale et d'exploitation des données personnelles des lecteurs.

C'est de ce constat qu'est né le projet Ex-Libris : une application de suivi de lecture pensée comme un véritable réseau social pour les lecteurs. L'objectif n'est pas de reconstruire un catalogue de livres, un travail déjà accompli et maintenu par la plateforme collaborative Open Library, qui référence plusieurs millions d'ouvrages en accès libre. Ce qu'Open Library ne propose pas, en revanche, c'est la dimension humaine de la lecture : qui a lu quel livre, quand, avec quel avis, et quelles recommandations peuvent en être tirées. C'est précisément cette couche que notre application vient ajouter.

Concrètement, Ex-Libris doit permettre à un utilisateur de créer un compte, de rechercher un livre et d'en consulter la fiche détaillée, de l'ajouter à sa bibliothèque personnelle avec un statut de lecture, de le noter et d'en rédiger une critique. Au-delà de cette gestion individuelle, l'application intègre une dimension sociale : chaque utilisateur peut consulter le profil d'un autre lecteur, s'abonner à son activité, et bénéficier d'un système de recommandation construit à partir des affinités entre utilisateurs.

D'un point de vue technique, le projet repose sur une API REST développée avec FastAPI, exploitant les données de l'API publique Open Library et s'appuyant sur une base de données PostgreSQL pour la persistance des données propres à notre application. Le langage retenu est Python, dans une logique de programmation orientée objet.

Ce dossier présente dans un premier temps le cahier des charges du projet, les choix et arbitrages qui ont guidé nos réflexions, ainsi que l'organisation retenue au sein de notre équipe. Il détaille ensuite le modèle de conception élaboré pour répondre à ces besoins, à travers les différents diagrammes UML réalisés.

2 Cahier des Charges

Avant d’entrer dans le détail de la modélisation, cette partie revient sur le cadre dans lequel notre travail s’est inscrit durant cette première phase du projet. Elle présente d’abord la manière dont notre équipe s’est organisée dès les premiers jours : la répartition des rôles, les outils choisis pour communiquer et collaborer, ainsi que le planning que nous nous sommes fixé pour tenir les différentes échéances imposées par le calendrier du projet.

Elle revient ensuite sur notre compréhension du sujet et des contraintes qui en découlent. Le fonctionnement d’Ex-Libris dépend en grande partie de la source de données sur laquelle nous nous appuyons, Open Library : il nous a donc fallu comprendre en détail la structure de cette base externe avant de pouvoir décider ce que notre propre application devait en retenir, en dupliquer ou en interroger à la volée. Cette compréhension a directement conditionné les choix présentés dans le reste du dossier, notamment au niveau du modèle de données et des diagrammes de conception.

Enfin, cette partie détaille les besoins fonctionnels de l’application à travers le diagramme de cas d’utilisation et les diagrammes d’activité, qui traduisent concrètement les différentes actions que pourront réaliser nos utilisateurs, ainsi que les enchaînements logiques entre elles.

2.1 Présentation d’Open Library



Notre application repose entièrement sur les données fournies par Open Library, une base collaborative et gratuite qui référence plusieurs millions d’ouvrages. Il s’agit d’une API web : on l’interroge en construisant des adresses (URLs) spécifiques, et elle nous renvoie les données demandées au format JSON.

Besoin	URL type	Exemple
Rechercher des livres	<code>search.json?q=...</code>	<code>openlibrary.org/search.json?q=le+petit+prince</code>
Voir une œuvre précise	<code>/works/{id}.json</code>	<code>openlibrary.org/works/OL10263W.json</code>
Voir une édition précise	<code>/books/{id}.json</code>	<code>openlibrary.org/books/OL7366018M.json</code>
Voir un auteur précis	<code>/authors/{id}.json</code>	<code>openlibrary.org/authors/OL31901A.json</code>
Voir la définition officielle d’un type	<code>/type/{nom}</code>	<code>openlibrary.org/type/work</code>

Pour bien comprendre comment exploiter ces données, il est essentiel de saisir la façon dont Open Library organise ses informations. La base distingue en réalité trois notions différentes qui, dans le langage courant, sont toutes désignées par le mot “livre” :

- L’auteur (Author) : la personne ayant écrit l’œuvre.

2.2 Diagramme de cas d'utilisation

Figure 11 – Diagramme de cas d'utilisation

2.3 Diagramme d'activité

3 Organisation du projet

3.1 Organisation du groupe

Dès le début du projet, nous avons désigné Anne-Camille Tampet comme cheffe de projet et Cécile Locatelli comme leader technique, en veillant, conformément aux conseils donnés en début de projet à ne pas cumuler ces deux rôles sur une même personne.

Pour communiquer, nous avons créé un groupe sur Teams, sur lequel chacun peut partager ses avancées, poser une question ou signaler un point de blocage dès que le besoin s'en fait sentir. C'est notre principal canal d'échange, sur lequel nous discutons régulièrement. En parallèle, nous avons mis en place un dépôt partagé sur GitHub, accessible à tous les membres du groupe, qui nous sert à centraliser et à distribuer l'ensemble des fichiers importants du projet.

Concernant la répartition du travail, notre organisation a surtout été pensée pour cette première phase d'analyse. Chaque membre du groupe s'est vu confier la réalisation d'un diagramme ou d'une partie spécifique du dossier, de façon à ce que chacun puisse s'investir concrètement dans la réflexion et la conception du projet. Cette répartition individuelle ne nous a cependant pas empêchés de travailler de façon très collective. Nous avons pris l'habitude de nous retrouver au minimum deux fois par semaine pour faire le point ensemble. Ces moments nous permettaient de relire et de discuter les diagrammes réalisés par chacun, de nous assurer que tout le monde comprenait bien les choix faits par les autres et de les faire évoluer si nécessaire. Cette phase de discussion collective a été particulièrement importante en tout début de projet. Elle nous a permis de poser une base commune solide et de nous assurer que tout le monde partageait la même vision de l'application avant d'avancer plus en détail sur la partie codage.

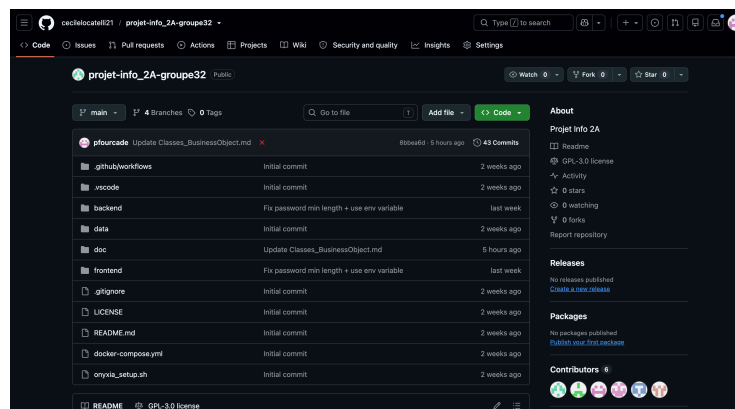


Figure 11 – Dépôt Github

3.2 Diagramme de Gantt

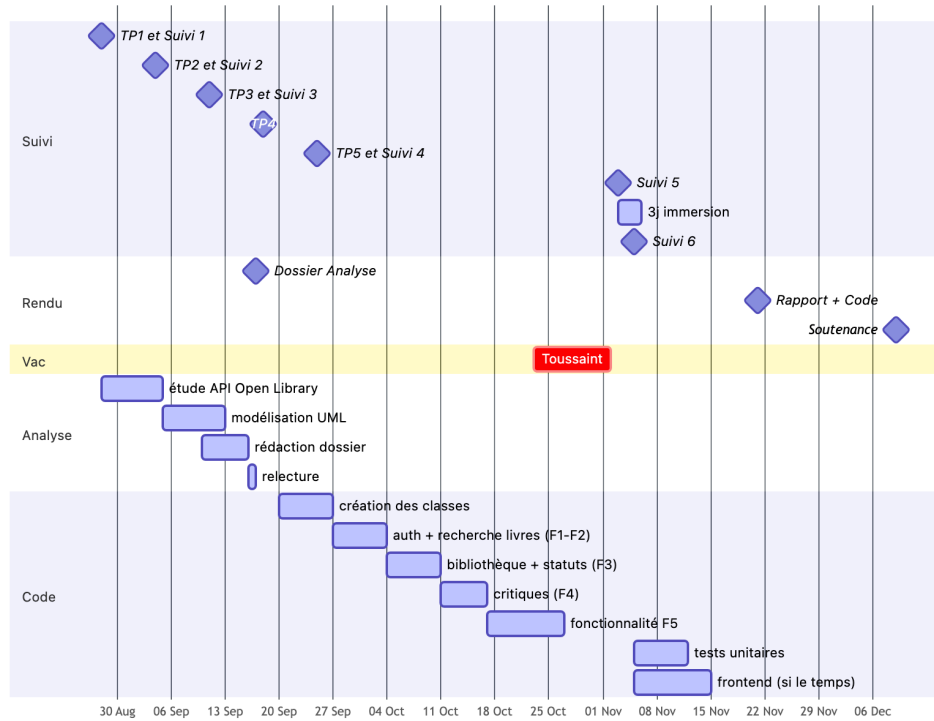


Figure 11 – Diagramme de Gantt

Notre planning se découpe en deux grandes phases. D'abord la phase d'analyse, de fin août à fin septembre : nous commençons par étudier l'API Open Library, puis nous réalisons l'ensemble de la modélisation UML afin de bien comprendre comment notre application va s'articuler et comment la construire.

Une fois cette base posée, nous passons à la phase de code, d'octobre à début novembre. Nous avons choisi de développer les fonctionnalités par ordre de difficulté croissante : d'abord les classes de base, puis l'authentification et la recherche de livres (F1-F2), la bibliothèque avec les statuts de lecture (F3), les critiques (F4), et enfin notre fonctionnalité F5 (les recommandations), qui nécessitera davantage de temps du fait de sa complexité. Les tests unitaires et le frontend interviendront en dernier, le frontend n'étant pas prioritaire, sera développé uniquement s'il nous reste du temps.

4 Maquette de l'application

4.1 Diagrammes BDD

4.2 Diagramme de classes

4.2.1 Classes Business Object

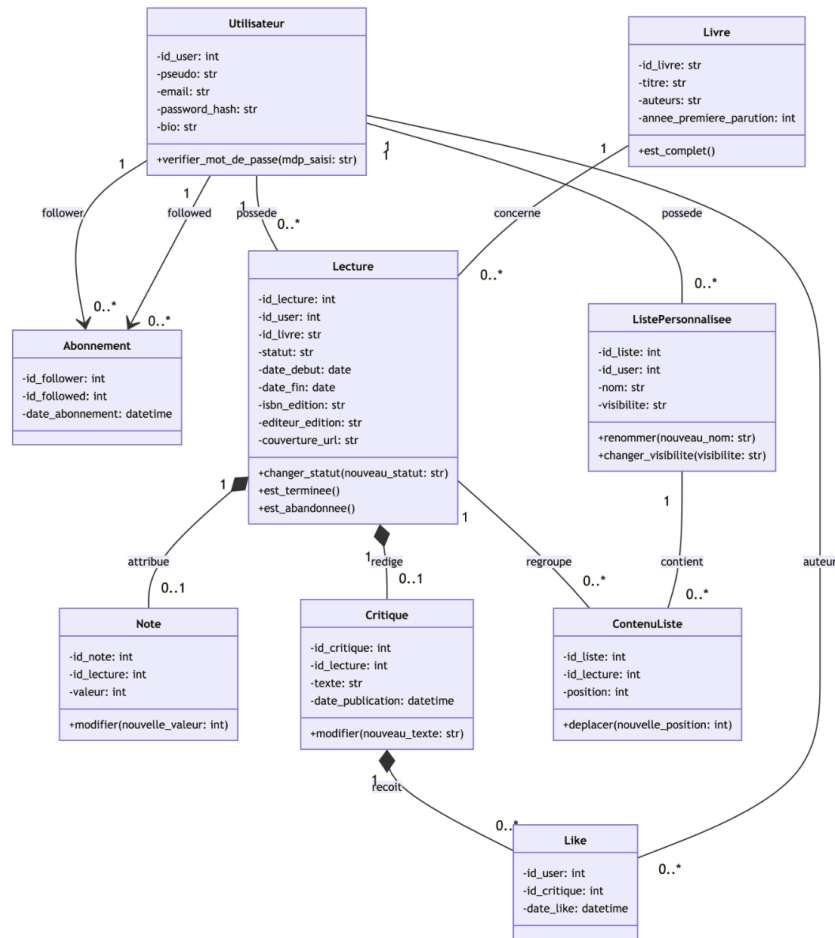


Figure 11 – Diagramme de classe : Business Object